

Display · adopted unit folders, shown and accepted on the page

1 · THE SAME FOLDER SHAPE AS A PAPER, AT A NEW ADDRESS

The plugin's full tree: one display/ per page, one folder per unit, every unit the same shape.  A page keeps its figures in the same folder shape a paper uses, so nobody has to learn a second one. The tree above is copied from the display family's own rules file, display-unit-output-contract.md, word for word. The folders split in two. intake/ and recipe/ are yours to write, filled by people and by drawing skills. assets/ and preview.pdf are rebuilt for you from those two. So a rebuild may replace the built half, and it never touches the half you wrote. The one page-side change is the address. The unit sits inside the page's own display/ folder, so when the page moves, its figures move with it.

2 · EVERY UNIT SHOWS AS A CARD, AND A SENTENCE CAN POINT AT IT

The tab: what a reader sees when they open  Display on a page.  Each unit shows as one card in the right pane, and naming its id in a sentence turns that id into a button. The tab is built the way the Slides tab is. A small button in the page's toolbar asks the server for this page's units. The server then writes a view that frames each unit's preview.pdf.  builds preview.pdf again from preview.tex. It never reruns the recipe, never changes the inputs, and never changes a yes. A unit is EVIDENCE. Name a unit's id in plain text, and that id turns into a small button in the sentence. Click it, and the unit's card opens with its picture and its yes-or-no state. There is a second way to attach the same evidence, made for machines. It is a > Display: line under a sentence, beside the > Citation: and > Value: lines. That way a program can add evidence without ever editing a person's words. You may point at a unit that has no picture yet, in either form. The button then says a drawing is still owed. The exports carry the evidence too. The LaTeX PDF prints a named unit as a real figure, right after the paragraph that names it. The Word file puts the picture in with its caption, and adds a  comment note on the sentence.

3 · THREE HANDS TOUCH A UNIT, AND ONLY ONE MAY SAY YES

Who writes what: three hands touch one unit, and each stays in its own half.  Drawing skills fill the folder, but the yes on a picture is always a person's click. The drawing skills already belong to the display family, and this page adds none of its own. The accepted: row is the one rule with teeth. A  there means a person looked at one exact picture and said yes. A machine that ticks it has faked a judgment, not saved time, so no machine may tick it. The kinds, and who draws them: one door sends the ask to five drawing skills, four scripted and one written by hand. A data-driven skill draws numbers, and it may not go back to the raw data. It reads only the small approved copy in intake/. A concept-driven skill draws an idea, so it holds no numbers at all, and its input is the sketch itself. In the  row a person draws by hand: they type the picture as .tex code, and that is how both units on this page were drawn. It gets its own row because the drawing is TeX code, so it uses the document's own fonts and math, and float.tex can pull the file in as it is. The skill is named after that way of drawing, not after one package, so haipipe-display-tex covers a TikZ figure, an algorithm block, and a math display (JL 260816). Such a unit still owes a built assets/figure.pdf. A document that pulls the code in rarely has the same setup lines at the top of the file. Drawing a unit again from its recipe depends on the kind. A python figure reruns its script, a diagram redraws from its spec, and an AI picture replays its saved prompt.

4 · THE ORDER OF THE WORK, AND WHICH STEPS NEED A PERSON

The five steps: from approved numbers to a picture a person accepts, and who moves each step.  Nothing is drawn before

a person approves the numbers, and nothing is finished before a person looks. Drawing never comes first. With no approved inputs there is nothing safe to draw, and the intake rules exist to refuse exactly that. Steps ① ③ ⑤ belong to a person, and steps ② ④ belong to the machine.

Qpf5-Display1 (Figure 1) draws that split of hands.¹

1 · haipipe

Display: QPf5-Display1-pipeline-tikz ·
kind=figure · label=fig:qpf5-pipeline

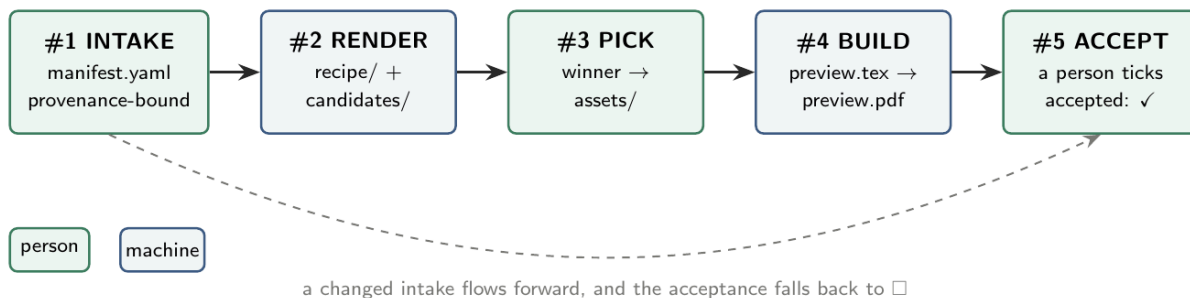


Figure 1. The display pipeline: five steps from approved intake to an accepted render. Green steps belong to a person, blue steps to machinery, and a changed intake returns the acceptance to open.

The tab's  reruns step ④'s build alone, never step ②'s

drawing skill. If the inputs change, the work runs forward

again from ② to ⑤. The old yes drops back to , because the picture a person agreed to no

longer exists. Where step ①'s numbers come from: each hop points at the one before it by id,

never by a copy. A probe is a question card. The page asks its question once, the answer lands in

the shared task list, and after that everyone points at the card's id instead of asking again. So

intake/ is not a second copy of the data. The task's own output stays the one source, and the

manifest only points at it, by id. A unit names its evidence by id, never by a file path, and it never

types a number by hand. QPf5-Display2 (Figure 2) draws that move: ask once, point at it twice.²

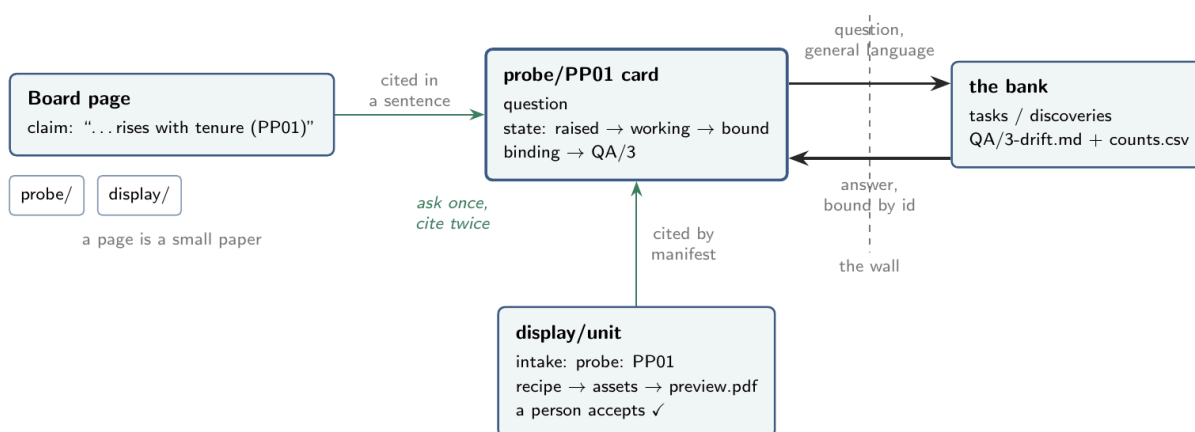


Figure 2. A page as a small paper: the probe plugin asks a question once, the bank answers behind the wall, and both the display intake and the page's own sentence cite the same card by id.

The slides plugin already works this way on QPf3. Its deck asks for an id, a small helper turns the id into the file's real path, and the template holds no digits at all. So a hand-typed number has nowhere to live. That id lookup already reaches units under a page, because the citation index finds a unit by id wherever the unit sits. Aim A1 asks for the other half. The build adapter should accept a page's display/ as a unit root, so --check can report an out-of-date page unit instead of hiding it.